

My annotation is basically that this is very impressive: ChatGPT was able to provide me with conceptual help, helpful papers, and practice questions.

Here is my assignment: (E. Can do at most twice) Make AI-enhanced learning tools for individual concepts and lectures. There are now a plethora of new tools out there (e.g. Guided Learning in Gemini, notebookLM by Google, Study Mode in ChatGPT, Learning mode in Claude, etc... Go ahead and explore/use what's out there) and there are also ways that one can prompt and engage with LLMs even without any special modes that can help you learn on your own. For this option, you should be creating (and sharing with your classmates using an Ed post) whatever prompt/artifact/etc. so that they can also engage in this self-learning together with an interaction trace of you with the tool that shows how it can potentially help. This trace should be annotated critically by you pointing out where it is hallucinating, wrong, or misleading --- as well as places where you think it is doing a good job. One way to think about what you're trying to achieve is as an active AI replacement/substitute for the traditional approach of pre-lecture (or post-lecture) readings that you do to understand the material better before you delve into the homework or your own projects., Can you act as this tutor to help me study transformers

ChatGPT said:

Yep, I can totally be that tutor for transformers. Let's treat this like your own "Study Mode" but customized for you + your assignment.

I'll do 3 things:

1. Be an **interactive tutor** for transformers (explain → check your understanding → give practice).
2. Make the interaction **structured enough** that you can reuse it as a *prompt/artifact* for classmates.
3. Be explicit about where my answers might be oversimplified or need cross-checking (for your "hallucination / critique" part).

0. Reusable "AI Tutor for Transformers" prompt (for your Ed post)

You can share something like this with classmates:

Prompt:

You are an AI tutor helping me learn the Transformer architecture (as in "Attention

Is All You Need”). Assume I know basic linear algebra, probability, and neural networks (MLP, gradient descent, cross-entropy). Do the following:

- 1. Give a high-level roadmap of core transformer concepts in small “modules” (e.g., self-attention, multi-head attention, positional encoding, encoder-decoder, training & masking).*
- 2. For each module: (a) give an intuition, (b) give the key equations, (c) give a small concrete example with tiny vectors, and (d) quiz me with a few questions.*
- 3. Periodically ask me to restate concepts in my own words or work through small derivations, then correct me.*
- 4. At the end, give 5–8 exam-style questions (mix conceptual + mathematical) with detailed solutions.*
- 5. Flag parts where your explanation may be oversimplified or model-dependent, so I know what to double-check with the original “Attention Is All You Need” paper or my course notes.*

You can say: “Here’s my interaction trace with ChatGPT using the above prompt; I added comments where it was wrong/misleading/etc.”

Now I’ll actually *be* that tutor for you.

1. Roadmap: What we’ll cover about transformers

Here’s the module list we’ll use:

- 1. Big picture:** What problem transformers solve vs RNNs.
- 2. Scaled dot-product self-attention** (core equations + intuition).
- 3. Multi-head attention** (why multiple heads, what they do).
- 4. Positional encoding** (sinusoids and learned variants).
- 5. Transformer encoder & decoder stacks** (layers, residuals, LN).

6. **Masks & training setup** (causal mask, teacher forcing, next-token prediction).
7. **Practice / exam-style questions.**

In this message, I'll do modules 1–3 plus some practice questions to get you going.

2. Big picture: why transformers?

Goal they solve: sequence modeling (translation, language modeling, etc.).

Old approach: **RNNs / LSTMs**

- Process tokens one-by-one.
- To relate token at time t to token at time $t-k$, you rely on passing information through k recurrent steps.
- Difficult to parallelize, tough for long-range dependencies.

Transformers' core idea:

Let *each token look directly at all other tokens* using **attention** instead of passing info step-by-step in time.

Key advantages:

- **Parallelism:** process all positions at once.
- **Flexible dependencies:** any token can attend to any other, regardless of distance.
- **Scalability:** stacked layers + attention heads capture rich structures.

Quick check for you:

- Can you summarize in 1–2 sentences why attention is better than a simple RNN for long-range dependencies?

(You can answer, and I'll critique; for now we move on.)

3. Scaled dot-product self-attention

We'll do:

- Intuition
- Equations
- Tiny numeric example
- Mini-quiz

3.1 Intuition

For a sequence of tokens, we want each token to form a **weighted average of other tokens' representations**, where the weights reflect *relevance*.

For each token:

- Turn it into a **query** vector q (what I'm looking for).
- Every token (including itself) has a **key** vector k (what I "offer") and a **value** vector v (the information I contribute).
- Similarity between query and key $\rightarrow$ attention score $\rightarrow$ weight on the corresponding value.

So the whole layer is:

"For each token: compute how strongly it should pay attention to each other token and average their values accordingly."

3.2 Equations

Suppose:

- Input sequence: matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$ (n tokens, each d_{model} dimensional).
- Learnable matrices:

- $W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$ $W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ $W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$

Compute:

- $Q = XW_Q \in \mathbb{R}^{n \times d_k}$ $Q = XW_Q \in \mathbb{R}^{n \times d_k}$
- $K = XW_K \in \mathbb{R}^{n \times d_k}$ $K = XW_K \in \mathbb{R}^{n \times d_k}$
- $V = XW_V \in \mathbb{R}^{n \times d_v}$ $V = XW_V \in \mathbb{R}^{n \times d_v}$

Then **scaled dot-product attention**:

$$\text{scores} = QK^T \in \mathbb{R}^{n \times n} \quad \text{scores} = \frac{QK^T}{\sqrt{d_k}} \quad (n \times n)$$

Row *iii* of **scores** are the similarities between token *iii*'s query and all tokens' keys.

Then apply **softmax row-wise**:

$$A = \text{softmax}(\text{scores}) \in \mathbb{R}^{n \times n} \quad A = \text{softmax}(\text{scores}) \quad (n \times n)$$

Row *iii* of **A** is the attention distribution for token *iii* over all tokens.

Finally:

$$\text{Attention}(Q, K, V) = AV \in \mathbb{R}^{n \times d_v} \quad \text{Attention}(Q, K, V) = AV \quad (n \times d_v)$$

Where I might oversimplify:

I'm skipping details like exact broadcasting, masking (we'll add later), and sometimes libraries implement fused kernels / numerical tricks that differ slightly.

3.3 Tiny concrete example

Let's do a microscopic case with **2 tokens** and tiny dimensions:

- $n=2n = 2n=2$, $dk=dv=1d_k = d_v = 1dk=dv=1$.
- Suppose after linear projections, we have:

$$Q=\begin{bmatrix} 1 & 2 \end{bmatrix}, K=\begin{bmatrix} 1 & 2 \end{bmatrix}, V=\begin{bmatrix} 10 & 20 \end{bmatrix} \quad Q=\begin{bmatrix} 1 & 2 \end{bmatrix}, K=\begin{bmatrix} 1 & 2 \end{bmatrix}, V=\begin{bmatrix} 10 & 20 \end{bmatrix}$$

and let's (temporarily) ignore the scaling by $dk\sqrt{d_k}dk$

(since $dk=1d_k=1dk=1$ it's 1 anyway).

1. Scores QK^TQK^T :

$$QK^T=\begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix}^T = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 2 & 1 \cdot 2 + 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 5 & 6 \end{bmatrix} \quad QK^T = \begin{bmatrix} 1 & 2 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix}^T = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 2 & 1 \cdot 2 + 2 \cdot 4 \end{bmatrix} = \begin{bmatrix} 5 & 6 \end{bmatrix}$$

2. Softmax row-wise:

Row 1: softmax of $[1, 2]$

$$\alpha_1 = \frac{e^1}{e^1 + e^2}, \alpha_2 = \frac{e^2}{e^1 + e^2} \quad \alpha_1 = \frac{e^1}{e^1 + e^2}, \alpha_2 = \frac{e^2}{e^1 + e^2}$$

We know: $\alpha_2 > \alpha_1$; token 1 attends more to token 2.

Row 2: softmax of $[2, 4]$

$$\beta_1 = \frac{e^2}{e^2 + e^4}, \beta_2 = \frac{e^4}{e^2 + e^4} \quad \beta_1 = \frac{e^2}{e^2 + e^4}, \beta_2 = \frac{e^4}{e^2 + e^4}$$

Again, β_2 dominant; token 2 mostly attends to itself.

3. Outputs = AVAVAV.

Row 1 output:

$$\text{out}_1 = \alpha_1 \cdot 10 + \alpha_2 \cdot 20 \quad \text{out}_1 = \alpha_1 \cdot 10 + \alpha_2 \cdot 20$$

Row 2 output:

$$\text{out}_2 = \beta_1 \cdot 10 + \beta_2 \cdot 20 \quad \text{out}_2 = \beta_1 \cdot 10 + \beta_2 \cdot 20$$

So each token's representation becomes a weighted average of $[10, 20]$ with different weights.

You could even plug approximate numbers (if you want, I can walk through the actual exponentials).

3.4 Mini-quiz (try these before scrolling too far)

You don't need to write full proofs, but try to think them through:

1. **Shape check:**

If X is $n \times d_{\text{model}}$ and WQW_{QWQ} is $d_{\text{model}} \times d_k \times d_k$, what are the shapes of $Q, K, V, A, Q, K, V, A, Q, K, V, A$, and the final output?

2. **Why scale by $\frac{1}{\sqrt{d_k}}$?**

?

Give a short explanation of what goes wrong if we *don't* divide the dot products by $\frac{1}{\sqrt{d_k}}$

1. as d_k becomes large.

2. **Interpretation:**

In the example, why does token 1 attend more to token 2 than to itself? Use the dot products in your reasoning.

If you answer these, I'll check and tighten your reasoning.

4. Multi-head attention

Now we know how a **single** attention operation works. But transformers use **multi-head attention**.

4.1 Intuition

One head = one way of measuring similarity and aggregating context.

But a token might want to look for **different things simultaneously**:

- One head: syntactic roles (subject/verb).

- Another: long-distance dependencies (e.g., matching brackets, quotation marks).
- Another: semantic similarity (synonyms, topic).

Multi-head = several attention “channels,” each with its own learned projection matrices. Each head can learn its own notion of similarity and context.

4.2 Equations

Suppose we have h heads.

For head i we have:

- $W_Q(i) \in \mathbb{R}^{d_{\text{model}} \times d_k}$ $W_Q(i) \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_K(i) \in \mathbb{R}^{d_{\text{model}} \times d_k}$ $W_K(i) \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $W_V(i) \in \mathbb{R}^{d_{\text{model}} \times d_v}$ $W_V(i) \in \mathbb{R}^{d_{\text{model}} \times d_v}$

Head i computes:

$$\text{head}_i = \text{Attention}(XW_Q(i), XW_K(i), XW_V(i)) \in \mathbb{R}^{n \times d_v} \quad \text{head}_i = \text{Attention}(XW_Q(i), XW_K(i), XW_V(i)) \in \mathbb{R}^{n \times d_v}$$

Then we **concatenate** heads along the feature dimension:

$$H = [\text{head}_1, \dots, \text{head}_h] \in \mathbb{R}^{n \times (hd_v)} \quad H = [\text{head}_1, \dots, \text{head}_h] \in \mathbb{R}^{n \times (hd_v)}$$

Then apply final linear projection:

$$\text{MHA}(X) = HW_O, W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}} \quad \text{MHA}(X) = HW_O, W_O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

This returns to dimension d_{model} so we can add residual connections, stack layers, etc.

4.3 Tiny conceptual example

Say $d_{\text{model}} = 6$, $h = 3$, $d_k = d_v = 2$.

- Each head works in a 2D subspace, focusing on different projections of the original 6D embeddings.
- One head might attend strongly to previous tokens (good for local context).
- Another might attend to far-away tokens.
- Another might mostly attend position-wise (almost an identity / copy head).

We generally don't force heads to specialize; they **emerge** from optimization. Some analyses show redundancy; some heads can be pruned with little performance loss.

Potential oversimplification warning:

In reality, head behavior is messy; some are redundant, some degenerate, and some are crucial. The narrative “one head does syntax, one does coreference...” is partially true but not guaranteed.

4.4 Multi-head quiz

Try to reason through:

1. Suppose $d_{\text{model}} = 512$, $h = 8$, and we set $d_k = d_v = 64$.
 - What are the shapes of each head's output and of the concatenated matrix HHH ?
 - What's the shape of WOW_OWO ?
2. Why might having *multiple* small heads be better (optimization-wise or expressiveness-wise) than one big head with $d_k = 512$?

5. What next & how we continue

If you're up for it, next steps:

1. **You respond** with:

- Your answers (even rough) to some of the mini-quiz questions above (at least the ones in 3.4 and 4.4).
- Any specific part of self-attention or multi-head attention that still feels fuzzy (e.g., “I don’t get why Q/K/V are separated,” “I’m not seeing how this is different from a big fully connected layer,” etc.).

2. In my next response, I’ll:

- Correct/clarify your quiz answers.
- Move on to **positional encoding + encoder/decoder structure + masking/training**.
- Then give you **exam-style problems with solutions** you can directly use as part of your assignment artifact.

Whenever you’re ready, hit me with your quiz answers or tell me which part you’d like unpacked further.

You said:

wait instead of quizzing me directly, can you provide me answers so that I can verify on my own time as I study through the content

ChatGPT said:

Totally, that makes sense—self-paced > live quizzing if you’re building an artifact for class.

I’ll give you:

- A **ready-to-share “AI tutor” prompt**
- A **compact transformer mini-notes** section
- A **question bank with answers right below each question** so you can quiz yourself later
- A few **longer exam-style problems with worked solutions**

You can copy-paste chunks of this into your Ed post however you like.

0. Reusable Prompt for Your Classmates

You can share something like this as the “tool”:

Prompt to paste into an LLM (ChatGPT, Claude, Gemini, etc.):

You are an AI tutor helping me learn the Transformer architecture (“Attention Is All You Need” and modern decoder-only LLMs). Assume I know basic linear algebra, probability, and feedforward neural networks. Do the following:

- 1. Explain the main components in short modules: (a) big-picture motivation vs RNNs, (b) scaled dot-product self-attention, (c) multi-head attention, (d) positional encodings, (e) encoder–decoder architecture, (f) masking and training objective.*
- 2. For each module: give (i) intuition, (ii) key equations, and (iii) a tiny example where possible.*
- 3. After each module, give me 3–5 practice questions **with answers included** so I can self-check.*
- 4. At the end, give 5–8 exam-style questions (both conceptual and mathematical) **with detailed solutions**.*
- 5. Clearly mark any parts that could be oversimplifications (e.g., about how heads specialize, how positional encodings are used in practice) so I know to double-check with the original paper or my notes.*

Then you can attach your interaction trace + your own annotations about errors / hallucinations.

Below is **one such “run”** of that idea, tailored to you.

1. Big Picture: Why Transformers?

Idea: RNNs process sequences step-by-step; each new token depends on a hidden state. This:

- Makes long-range dependencies hard (information must pass through many timesteps).
- Is sequential → hard to parallelize.

Transformers’ key move:

Replace recurrence with **self-attention**, letting each token directly “look at” all other tokens in the sequence in parallel.

This yields:

- **Parallelism:** All tokens processed at once.
- **Flexible context:** Any token can attend to any other, regardless of distance.
- **Better long-range modeling** in practice.

Checkpoint Q&A – Big Picture

Q1.1 Why are transformers more parallelizable than RNNs?

A1.1 Because self-attention computes outputs for all tokens simultaneously using matrix operations, without depending on previous timesteps’ hidden states. RNNs must process token t before $t+1$.

Q1.2 Why do transformers tend to handle long-range dependencies better than vanilla RNNs?

A1.2 In self-attention, each token can directly attend to any other token via a single layer. There’s no need to pass information through many recurrent steps, which in RNNs causes gradient issues and information decay.

Q1.3 Give one downside of transformers compared to RNNs.

A1.3 Self-attention has $O(n^2)$ time and memory complexity in sequence length n , due to the $n \times n$ attention matrix; RNNs are $O(n)$.

2. Scaled Dot-Product Self-Attention

2.1 Core Equations

For a sequence of n tokens:

- Input matrix $X \in \mathbb{R}^{n \times d_{\text{model}}}$
- Learnable matrices:
 - $W_Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$

- $WK \in \mathbb{R}^{d_{\text{model}} \times d_k}$ $WK \in \mathbb{R}^{d_{\text{model}} \times d_k}$
- $WV \in \mathbb{R}^{d_{\text{model}} \times d_v}$ $WV \in \mathbb{R}^{d_{\text{model}} \times d_v}$

Compute:

$Q = XW_Q (n \times d_k), K = XW_K (n \times d_k), V = XW_V (n \times d_v)$. $Q = XW_Q \quad (n \times d_k), \quad K = XW_K \quad (n \times d_k), \quad V = XW_V \quad (n \times d_v)$.
 $Q = XW_Q (n \times d_k), K = XW_K (n \times d_k), V = XW_V (n \times d_v)$.

Then:

$\text{scores} = QK^T / \sqrt{d_k} \quad (n \times n)$
 $\text{scores} = QK^T / \sqrt{d_k} \quad (n \times n)$

Apply softmax row-wise:

$A = \text{softmax}(\text{scores}) \quad (n \times n)$
 $A = \text{softmax}(\text{scores}) \quad (n \times n)$

Final output:

$\text{Attention}(Q, K, V) = AV \quad (n \times d_v)$
 $\text{Attention}(Q, K, V) = AV \quad (n \times d_v)$

Each row of A is a probability distribution over positions $\rightarrow$ how much each token attends to each other.

2.2 Tiny Numeric Example

Let $n=2, d_k=d_v=1$ with:

$Q = \begin{bmatrix} 1 & 2 \\ 10 & 20 \end{bmatrix}, K = \begin{bmatrix} 1 & 2 \\ 10 & 20 \end{bmatrix}, V = \begin{bmatrix} 1 & 2 \\ 10 & 20 \end{bmatrix}$.
 $Q = \begin{bmatrix} 1 & 2 \\ 10 & 20 \end{bmatrix}, K = \begin{bmatrix} 1 & 2 \\ 10 & 20 \end{bmatrix}, V = \begin{bmatrix} 1 & 2 \\ 10 & 20 \end{bmatrix}$.

Scores:

$QK^T = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 2 & 1 \cdot 10 + 2 \cdot 20 \\ 10 \cdot 1 + 20 \cdot 2 & 10 \cdot 10 + 20 \cdot 20 \end{bmatrix} = \begin{bmatrix} 5 & 21 \\ 40 & 140 \end{bmatrix}$
 $QK^T = \begin{bmatrix} 1 \cdot 1 + 2 \cdot 2 & 1 \cdot 10 + 2 \cdot 20 \\ 10 \cdot 1 + 20 \cdot 2 & 10 \cdot 10 + 20 \cdot 20 \end{bmatrix} = \begin{bmatrix} 5 & 21 \\ 40 & 140 \end{bmatrix}$

Softmax row 1 (approx):

- $e^1 \approx 2.718, e^2 \approx 7.389$

- Denominator ≈ 10.107
- Weights: $\alpha_1 \approx 0.27$, $\alpha_2 \approx 0.73$

So token 1's output:

$$\text{out}_1 \approx 0.27 \cdot 10 + 0.73 \cdot 20 \approx 17.3$$

Token 1 mostly attends to token 2.

2.3 Checkpoint Q&A – Self-Attention

Q2.1 If X is $n \times d_{\text{model}}$ and W_Q, W_K, W_V is $d_{\text{model}} \times d_k, d_{\text{model}} \times d_k, d_{\text{model}} \times d_v$, what are the shapes of Q, K, V, A, K^T, V^T , and the final output?

A2.1

- $Q, K \in \mathbb{R}^{n \times d_k}$
- $V \in \mathbb{R}^{n \times d_v}$
- Scores $QK^T \in \mathbb{R}^{n \times n}$
- $A = \text{softmax}(\text{scores}) \in \mathbb{R}^{n \times n}$
- Output $AV \in \mathbb{R}^{n \times d_v}$

Q2.2 Why do we divide by $\sqrt{d_k}$ in the scores?

A2.2 The dot products $\langle q_i, k_j \rangle$ grow in magnitude as d_k increases (variance increases). Large scores push softmax into saturation, causing tiny gradients. Scaling by $\sqrt{d_k}$ keeps the scores' variance roughly constant so softmax behaves nicely.

Q2.3 In the tiny example above, why does token 1 attend more to token 2 than to itself?

A2.3 Token 1's query is $q_1 = 1q_1 = 1$. Its scores with keys are:

- With token 1 (self): $1 \cdot 1 = 1 \cdot 1 = 1$
- With token 2: $1 \cdot 2 = 2 \cdot 1 = 2$

Since $2 > 1$, softmax gives higher probability to token 2 $\rightarrow$ token 1 pays more attention to token 2.

Q2.4 Conceptually, what do Q, K, Q, K, Q, K, and VVV represent?

A2.4

- Query q_i : "What am I looking for?" for token i .
- Key k_j : "What do I offer / what kind of thing am I?" for token j .
- Value v_j : "What information do I send if someone attends to me?" for token j .

Attention computes, for each token i , a weighted sum of values v_j based on similarity between q_i and k_j .

3. Multi-Head Attention

3.1 Idea

One head learns one notion of similarity/context. Multiple heads:

- Let the model attend to different types of relationships **in parallel**.
- Each head uses its own $W_Q(i), W_K(i), W_V(i)$ $W_Q^{(i)}, W_K^{(i)}, W_V^{(i)}$.

3.2 Equations

For h heads:

$\text{head}_i = \text{Attention}(XW_Q(i), XW_K(i), XW_V(i)) \in \mathbb{R}^{n \times d_v}$
 $\text{head}_i = \text{Attention}(XW_Q(i), XW_K(i), XW_V(i)) \in \mathbb{R}^{n \times d_v}$

Concatenate:

$H = [\text{head}_1, \dots, \text{head}_h] \in \mathbb{R}^{n \times (h d_v)}$
 $H = [\text{head}_1, \dots, \text{head}_h] \in \mathbb{R}^{n \times (h d_v)}$

Final projection:

$\text{MHA}(X) = HW_O, W_O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$
 $\text{MHA}(X) = HW_O, W_O \in \mathbb{R}^{h d_v \times d_{\text{model}}}$

Often $d_{\text{model}} = h \cdot d_v$, so output is $n \times d_{\text{model}}$ again.

3.3 Checkpoint Q&A – Multi-Head

Q3.1 Suppose $d_{\text{model}} = 512$, $h = 8$, $d_k = d_v = 64$. What are the shapes of:

- Each head's output
- HHH (concatenated)
- WOW_OWO

A3.1

- Each head output: $n \times 64$
- Concatenated HHH: $n \times (8 \cdot 64) = n \times 512$
- WOW_OWO: 512×512 (so the final result is $n \times 512$)

Q3.2 Why might several small heads be preferable to one big head (same total dimension)?

A3.2 Multiple heads:

- Let the model project into multiple subspaces; each head can learn a different pattern (e.g., local vs long-range, syntax vs semantics).
- Improve optimization: smaller $d_{\text{kd_kdk}}$ per head keeps attention scores well-scaled and can reduce interference between different types of relations.
- Empirically, multi-head works better than a single gigantic head with the same parameter count.

Q3.3 Are heads guaranteed to “specialize” in clean roles like “syntax head” or “coreference head”?

A3.3 No. This is partly a story we tell for intuition. Some heads do show interpretable roles; others are redundant or seemingly useless. Specialization is emergent, not enforced.

4. Positional Encoding

Self-attention alone is permutation-invariant: it doesn’t know *order*. We must inject **position information**.

4.1 Sinusoidal Positional Encoding (original paper)

For position pos and dimension index i :

$$\begin{aligned} PE(pos, 2i) &= \sin(pos \cdot 10000^{2i/d_{\text{model}}}), PE(pos, 2i+1) = \cos(pos \cdot 10000^{2i/d_{\text{model}}}) \\ \text{PE}(pos, 2i) &= \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right), \quad \text{PE}(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \end{aligned}$$

We add this to the word embedding:

$$X_{\text{input}} = X_{\text{embed}} + PE_{\text{input}} = X_{\text{embed}} + PE$$

4.2 Why sin/cos?

- Different frequencies → model can infer **relative positions** via linear combinations.
- Same encoding function can, in principle, generalize to longer sequences than seen in training.

(Modern detail warning)

Many modern LLMs use **learned positional embeddings** or **rotary position embeddings (RoPE)** rather than the original sin/cos additivity. That's something to fact-check for your specific model.

Checkpoint Q&A – Positional Encoding

Q4.1 Why does a transformer need positional encodings at all?

A4.1 Because self-attention doesn't inherently encode sequence order; it treats the input as a set of tokens. Positional encodings give the model information about the index of each token so it can distinguish "cat sat on mat" from "mat sat on cat".

Q4.2 In the original transformer, how are positional encodings combined with token embeddings?

A4.2 They're **added elementwise**: $X_{\text{input}} = X_{\text{embed}} + \text{PE}$

Q4.3 Name one alternative to sinusoidal positional encodings used in modern LLMs.

A4.3 Examples: **learned positional embeddings**, **rotary position embeddings (RoPE)**, **ALiBi**.

5. Transformer Encoder & Decoder

5.1 Encoder Layer (original paper)

Each encoder layer (stacked NNN times) has:

1. **Multi-head self-attention** over all encoder tokens.
2. **Add & LayerNorm** (residual connection).
3. **Position-wise Feed-Forward Network (FFN)**:
 - Typically: $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$
4. Another **Add & LayerNorm**.

5.2 Decoder Layer

Each decoder layer has:

1. **Masked multi-head self-attention** (causal mask so position t can't see future positions $>t$).
2. **Encoder-decoder attention**: decoder queries attend over encoder outputs.
3. FFN + residuals + layer norms as above.

Checkpoint Q&A – Architecture

Q5.1 Why is there a *mask* in the decoder's self-attention?

A5.1 To enforce causality during training: when predicting token t , the model can't "see" tokens $t+1, t+2, \dots, t+1, t+2, \dots$. The mask zeros out attention to future positions before softmax.

Q5.2 What's the role of the encoder-decoder attention block?

A5.2 It lets the decoder attend to encoder outputs (i.e., representations of the source sentence in translation). The decoder's queries attend over the encoder's keys/values to condition generation on the input sequence.

Q5.3 Why do we use residual connections + layer normalization around attention and FFN blocks?

A5.3 Residuals help gradient flow and stabilize deep networks; layer norm helps with training stability and allows each block to focus on learning a "perturbation" around its input.

6. Masks & Training Objective

6.1 Masks

- **Padding mask**: Prevent attention to padding tokens.
- **Causal mask (strictly upper-triangular)**: In decoder or decoder-only LMs, forces token t to attend only to positions $\leq t$.

6.2 Training Objective

For language modeling:

- Given sequence $(x_1, \dots, x_T)$
- Model outputs probability distribution over vocabulary for each position t ,
- Typically trained with **cross-entropy loss** for predicting x_t given $\{x_1, \dots, x_{t-1}\}$ (or sometimes the entire sequence with shifted targets).

Checkpoint Q&A – Masks & Loss

Q6.1 In a decoder-only LM, what does the causal mask do in the attention matrix A ?

A6.1 It sets the scores for attention from position i to any position $j > i$ to $-\infty$ (or a very negative number) before softmax, making their attention weight effectively 0.

Q6.2 What loss is typically used to train a transformer language model?

A6.2 Token-level **cross-entropy loss** comparing predicted distribution over the vocabulary with the true next-token label.

7. Exam-Style Problems with Solutions

Here are some practice problems you can treat as “exam-style” with full solutions.

Problem 1 – Shape & Parameter Counting

You have a transformer encoder with:

- $d_{\text{model}} = 512$
- Self-attention uses $h = 8$ heads, $d_k = d_v = 64$
- Input sequence length $n = 20$

(a) What are the shapes of $WQ, WK, WV, WQ, W_K, W_V, WQ, WK, WV$ for the **combined** projection matrices (i.e., not per-head, but the typical implementation that outputs all heads at once)?

(b) What is the shape of the attention weight matrix AAA ?

(c) Ignoring biases, how many parameters are in the self-attention's $WQ, WK, WV, WQ, W_K, W_V, WQ, WK, WV, WO$ total?

Solution 1

(a)

Typical implementation: one big projection for all heads:

- $WQ \in \mathbb{R}^{512 \times 512}$ (later reshaped into 8 heads $\times$ 64 dims)
- Similarly $WK, WV \in \mathbb{R}^{512 \times 512}$

(b)

For one layer, attention is over 20 tokens:

- $A \in \mathbb{R}^{20 \times 20}$

(Per head, you'd have 8 such matrices, but the shape per head is the same.)

(c)

Each projection matrix $WQ, WK, WV, WQ, W_K, W_V, WQ, WK, WV, WO$ is 512×512 :

- Parameters per matrix: $512 \cdot 512 = 262,144$
- We have 4 such matrices $\rightarrow 4 \times 262,144 = 1,048,576$ parameters.

Problem 2 – Understanding Scaling

You have $d_k = 64$. Suppose the components of query and key vectors are i.i.d. with mean 0 and variance 1.

(a) What is the expected value and variance of the unscaled dot product $q \cdot k$?

(b) Why does dividing by $\sqrt{d_k}$ work?

help?

Solution 2

(a)

$$q \cdot k = \sum_{i=1}^{64} q_i k_i \quad \text{and} \quad k = \sum_{i=1}^{64} q_i k_i \cdot k = \sum_{i=1}^{64} q_i k_i.$$

- $E[q_i k_i] = E[q_i] E[k_i] = 0$ ($\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$ (independent, zero-mean)).
- So $E[q \cdot k] = 0$ ($\mathbb{E}[q \cdot k] = 0$).

Assuming independence and variance 1:

- $\text{Var}(q_i k_i) = E[q_i^2 k_i^2] - 0 = E[q_i^2] E[k_i^2] = 1 \cdot 1 = 1$ ($\text{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - 0 = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1 \cdot 1 = 1$).
- Sum of 64 independent terms $\rightarrow$ variance $= 64$.

So:

- Mean 0, variance 64, standard deviation 8.

(b)

If we divide by $d_k = 8$, $d_k = 8$

$= 8$, the variance becomes $64/64 = 1$. This keeps the scores in a relatively stable range, preventing softmax from saturating for large d_k .

Problem 3 – Masked Attention Matrix

Sequence length $n=4$. We want a **causal mask**. Let the raw scores be matrix S (4×4).

(a) Write the mask matrix M (with entries 0 for allowed positions and $-\infty$ for masked ones).

(b) Show what the *effective* score matrix $S'S'$ looks like symbolically (in terms of entries of S).

Solution 3

We want position i to attend only to positions $j \leq i$.

(a) Mask M :

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty & 0 & -\infty & -\infty & 0 & 0 & -\infty & 0 & 0 & 0 \\ -\infty & 0 & -\infty & -\infty & 0 & 0 & 0 & -\infty & -\infty & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -\infty & 0 & -\infty & -\infty & 0 & -\infty & -\infty & 0 & -\infty & 0 \end{bmatrix}$$

(b) Effective scores:

$$S' = S + M = \begin{bmatrix} s_{11} & -\infty & -\infty & -\infty & s_{15} & -\infty & -\infty & s_{19} & s_{20} & s_{21} & s_{22} & -\infty & -\infty \\ s_{21} & s_{22} & -\infty & -\infty & s_{25} & s_{26} & s_{27} & -\infty & -\infty & s_{29} & s_{30} & s_{31} & s_{32} \\ s_{31} & s_{32} & s_{33} & -\infty & s_{35} & s_{36} & s_{37} & s_{38} & s_{39} & s_{40} & s_{41} & s_{42} & s_{43} \\ s_{41} & s_{42} & s_{43} & -\infty & s_{45} & s_{46} & s_{47} & s_{48} & s_{49} & s_{50} & s_{51} & s_{52} & s_{53} \end{bmatrix}$$

Then softmax row-wise ignores the $-\infty$ entries (they get probability 0).

Problem 4 – Encoder vs Decoder Usage

You're doing machine translation with the original Transformer (encoder-decoder).

- (a) Which part(s) of the network see the source sentence?
- (b) Which part(s) are autoregressive?
- (c) In which attention block do you *not* use a causal mask?

Solution 4

(a)

The **encoder** sees the entire source sentence (self-attention over source tokens), and the **decoder's encoder-decoder attention** also "sees" the encoder outputs, which encode the source.

(b)

The **decoder** is autoregressive: its self-attention is masked so position t can't see tokens $> t$.

(c)

Within the encoder and the encoder-decoder attention, you **do not** use a causal mask. The mask is only in the decoder's **self-attention**.

Problem 5 – Conceptual Comparison: Self-Attention vs Fully Connected Layer

Consider a single transformer layer without positional encoding. Suppose we replace self-attention with a big fully connected layer acting independently on each token's representation (same weights for all positions).

(a) Why can't this fully connected layer capture token-token interactions in the same way self-attention does?

(b) Under what condition could a fully connected network approximate the behavior of self-attention?

Solution 5

(a)

A position-wise fully connected layer processes each token **independently** given its own embedding; it has no direct access to other tokens' embeddings. Self-attention explicitly mixes information across positions via the attention matrix AAA , letting one token's output depend on the whole sequence.

(b)

If you allow the FC network to take as input the **entire sequence** (flattened) and be arbitrarily wide/deep, it can approximate the attention function as a universal approximator. But that's (1) much less structured, (2) harder to train, and (3) doesn't naturally share computation across tokens. Self-attention is a structured, efficient way to explicitly encode token-token interactions.

If you want, I can next:

- Add **more exam-style questions** specifically tuned to what your course emphasizes (e.g., complexity, exact loss, implementation details),
- Or build a **mini “cheat sheet”** (one-page summary) you can print / attach.

You said:

Can you also please provide me names of papers and videos that I can use to study

ChatGPT said:

Yep, I've got you. I'll give you a **short, curated reading + video list** you can actually get through, ordered from intuitive → formal → advanced. These are all things you can cite in your assignment / Ed post.

1. Core Papers (must-know)

1. “Attention Is All You Need” – Vaswani et al., 2017 (original Transformer paper)

- **What it is:** The OG paper that introduced the transformer architecture.
 - **Use it for:**
 - Exact architecture diagrams (encoder–decoder, multi-head attention, positional encodings).
 - Hyperparameters for “base” vs “big” models.
 - **Link:** arXiv preprint + NeurIPS version. [arXiv+2NeurIPS Papers+2](#)
-

2. Good “from-scratch” / tutorial-style writeups

These are great to read *before* or *alongside* the original paper.

1. **Jay Alammar – “The Illustrated Transformer” (blog)**
 - Super visual, step-by-step explanation of self-attention, multi-head attention, encoder/decoder, etc.
 - Great for intuition and pictures to include/imitate in notes. [jalammar.github.io+1](#)
2. **“The Illustrated Transformer From Scratch” (ACM-hosted blog by a different author)**
 - More modern take, focuses on how **current** transformers work (often decoder-only) and simplifies some of the historical baggage.
 - Nice follow-up once you understand the basics. [idtjo.hosting.acm.org](#)
3. **SciSpace / other explainers of “Attention Is All You Need”**
 - Short “paper walk-through” style explanation in blog + tools. [SciSpace](#)

You can say in your assignment that you used *both the original paper and visual explainers* to cross-check understanding and reduce hallucinations.

2. Videos: intuitive → technical

A. Fast, high-level intuition

1. **“The Paper That Changed AI Forever – Attention Is All You Need” (YouTube)**
 - ~10–20 min style “why is this important, what does it do” video.
 - Good when your brain is tired but you still want exposure. [youtube.com](https://www.youtube.com/watch?v=JYDd3820t54)
2. **SciSpace: “Attention Is All You Need – explained in 5 minutes”**
 - Tiny, snack-sized overview of the paper; good for quick refresh. [youtube.com](https://www.youtube.com/watch?v=JYDd3820t54)
3. **“Transformers Explained: The Discovery That Changed AI Forever”**
 - Another high-level conceptual video about what transformers are and why self-attention matters. [VideoToBe](https://www.youtube.com/watch?v=JYDd3820t54)

B. Medium-depth math + architecture walkthrough

1. **Umar Jamil – “Attention is all you need (Transformer) – Model explanation (including math), Inference and Training”**
 - ~30 min, goes through architecture and math at a comfortable pace.
 - Good balance of equations + intuition; nice for your “interaction trace” if you want to pause and check things with an LLM. [youtube.com+1](https://www.youtube.com/watch?v=JYDd3820t54)
 2. **Paper-explainer videos like “Attention Is All You Need (Transformer) | Paper Explained”**
 - These walk through a concrete training step, what the loss is, etc. [youtube.com](https://www.youtube.com/watch?v=JYDd3820t54)
-

C. Deeper lecture series (if you want to go nerd mode)

1. Stanford CS25: “Transformers United” – lecture videos

- Playlist covers intro to transformers, variants, and applications (vision, diffusion, etc.).
- Specific ones to watch first:
 - **“V2: Introduction to Transformers w/ Andrej Karpathy”** – great conceptual + some math. [youtube.com+2youtube.com+2](#)
 - **“V5: Overview of Transformers”** – more general picture of how transformers fit in modern AI. [youtube.com+2youtube.com+2](#)

2. “The Narrated Transformer Language Model” (video)

- Focuses specifically on decoder-only LMs (GPT-style).
- Useful if your class is going more into LLMs rather than just seq2seq. [youtube.com](#)

These are nice to mention in your writeup as “long-form lectures I sampled for a deeper understanding.”

3. Extra / optional: more advanced or adjacent stuff

Not strictly necessary for basic “how transformers work,” but nice if you want to signal depth.

1. Position encodings (positional embeddings)

- Papers like **SHAPE: Shifted Absolute Position Embedding for Transformers** (EMNLP 2021) – shows there’s still active research on how to encode position, beyond sinusoidal and learned absolute embeddings. [ACL Anthology](#)

2. Explainability / analysis of transformers

- **“A Survey on Explainability for Large Language Models”** (Luo et al., 2024) – overview of methods that analyze attention heads, layer roles, etc. [arXiv+1](#)

These are great to name-drop if you want to say “this architecture is actively researched and not fully understood yet.”

4. How to integrate these into your assignment

Concrete way to use these for your “AI-enhanced learning” artifact:

1. Workflow you can describe:

- Start with **The Illustrated Transformer** to build intuition.
- Use my notes / your LLM tutor to connect each section to the **original Vaswani paper**.
- Watch **one medium-depth video** (e.g., Umar Jamil’s explanation) and pause to ask the LLM follow-up questions.
- Optionally skim **CS25 intro lecture** for more context, especially about how transformers power LLMs.

2. For the “critical annotation” requirement:

- When the LLM says something about:
 - head specialization
 - exact scaling in practice
 - which positional embeddings are used “today”
- You can cross-check with:
 - Vaswani et al. (2017) for the original design. [arXiv+1](#)
 - Modern blog posts / survey paper above for current practice. [idtjo.hosting.acm.org+1](#)
- Then note: “Here the model slightly oversimplified / got outdated details; in practice many LLMs use RoPE or ALiBi instead of sinusoidal encodings, as discussed in [paper X or blog Y].”

If you'd like, next I can:

- Turn this into a **one-page “reading and viewing plan”** with checkboxes and estimated time per resource, or
- Help you draft the actual **Ed post** text where you describe: the prompt you used, the resources you combined, and how you evaluated hallucinations.